



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA EN SOFTWARE

Análisis del Nivel Táctico del Sistema RetailMind:

Objetivos Tácticos por Departamento y Clasificación de Informes

(Simples - BDR / Compuestos - BD Columnar)

TA 11

***Materia:** Construcción de Software*

***Nombre:** Benites Pérez Dariem Alberto*

Sexto Semestre

Quevedo, Ecuador - 2026

1. Introducción

El sistema RetailMind concluyó la construcción de su nivel operativo: registro de las transacciones diarias de la PyME (ciclo completo de venta con pagos y facturación, ciclo de compra con aprobaciones, inventario con kardex, logística de despacho y devoluciones, soporte al cliente y marketing). El presente documento da el siguiente paso en la pirámide informacional de la organización: el análisis del nivel táctico.

Mientras el nivel operativo responde a la pregunta «¿qué transacción ocurre ahora?», el nivel táctico responde a «¿cómo se dirige y controla cada área?». Un objetivo táctico es una necesidad de información planteada por el jefe de cada departamento, orientada a la dirección, el seguimiento y el control de su área en el mediano plazo. A partir de esos objetivos se derivan los informes tácticos que el sistema debe producir.

1.1. Informes simples y compuestos

Cada informe táctico se clasifica según su costo de obtención sobre los datos:

- **Informe SIMPLE:** responde «¿cómo está esto ahora?». Se obtiene directamente de la base de datos relacional (BDR) con una consulta SELECT sencilla, sin agregaciones pesadas ni transformación temporal. *Ejemplo:* pedidos pendientes de despacho.
- **Informe COMPUESTO:** responde «¿cómo evoluciona o se agrega esto en el tiempo?». Requiere agregaciones sobre volúmenes históricos, comparaciones entre períodos o cruces de varias fuentes; ejecutarlo sobre la BDR transaccional la penalizaría, por lo que se procesa en la base de datos columnar, alimentada mediante un proceso ETL orquestado con Apache Airflow. *Ejemplo:* tendencia de ventas por mes y categoría.

1.2. Arquitectura híbrida del sistema real

RetailMind ya materializa esta separación con una arquitectura híbrida de dos motores: PostgreSQL como única base transaccional (BDR, base retailmind con aproximadamente 103 tablas operativas, seguridad por roles, RLS y triggers de negocio) y ClickHouse como base columnar dedicada exclusivamente a analítica. Los informes simples de este análisis se resuelven con consultas directas sobre PostgreSQL; los compuestos justifican el pipeline ETL (extracción desde la BDR, transformación y carga en ClickHouse) cuya orquestación con Airflow constituye la fase pendiente del proyecto.

Los objetivos tácticos se levantaron para los siete departamentos del negocio - Ventas, Compras, Inventario/Bodega, Logística/Despacho, Soporte y Gerencia/Dirección (que gestiona también Marketing) - y cada uno fue verificado contra el esquema real de la base de datos para garantizar que es alcanzable con los datos existentes.

2. Objetivos tácticos por departamento y clasificación de informes

La siguiente tabla consolida los objetivos tácticos provistos por cada jefatura departamental, la clasificación de su informe y la base de datos de la que se obtiene.

DEPARTAMENTO	OBJETIVO TÁCTICO	¿SIMPLE?	¿COMPUESTO?	FUENTE
VENTAS	Controlar ventas por vendedor (cumplimiento individual)	X	—	BDR (PostgreSQL)
	Monitorear pedidos por estado (pendientes, despachados, entregados)	X	—	BDR (PostgreSQL)
	Analizar tendencia de ventas por mes y categoría	—	X	BDColumnar (ClickHouse)
	Medir ticket promedio de compra por período	—	X	BDColumnar (ClickHouse)
COMPRAS	Ver órdenes de compra pendientes de aprobación	X	—	BDR (PostgreSQL)
	Controlar cuentas por pagar vigentes y vencidas	X	—	BDR (PostgreSQL)
	Analizar gasto de compra por proveedor por trimestre	—	X	BDColumnar (ClickHouse)
	Evaluar cumplimiento de tiempos de entrega por proveedor	—	X	BDColumnar (ClickHouse)
INVENTARIO / BODEGA	Identificar productos bajo stock mínimo	X	—	BDR (PostgreSQL)
	Consultar stock actual por bodega	X	—	BDR (PostgreSQL)
	Analizar rotación de inventario por categoría	—	X	BDColumnar (ClickHouse)
	Medir productos de baja/nula rotación por período	—	X	BDColumnar (ClickHouse)
LOGÍSTICA / DESPACHO	Ver pedidos preparados pendientes de despacho	X	—	BDR (PostgreSQL)
	Consultar envíos por transportista	X	—	BDR (PostgreSQL)
	Analizar tiempo promedio de entrega por transportista/zona	—	X	BDColumnar (ClickHouse)
	Medir tasa de devoluciones sobre despachos por período	—	X	BDColumnar (ClickHouse)
SOPORTE	Ver tickets abiertos por prioridad y estado	X	—	BDR (PostgreSQL)
	Consultar tickets asignados por agente	X	—	BDR (PostgreSQL)
	Analizar tiempo promedio de resolución por categoría por mes	—	X	BDColumnar (ClickHouse)
	Medir cumplimiento de SLA por período	—	X	BDColumnar (ClickHouse)
GERENCIA / DIRECCIÓN (INCLUYE MARKETING)	Consultar estado consolidado de pedidos del día	X	—	BDR (PostgreSQL)
	Listar cupones activos y promociones vigentes (marketing)	X	—	BDR (PostgreSQL)
	Analizar tasa de redención de cupones e impacto de promociones por período (marketing)	—	X	BDColumnar (ClickHouse)
	Analizar rentabilidad por categoría por período	—	X	BDColumnar (ClickHouse)
	Medir evolución de ingresos vs. costos mensual	—	X	BDColumnar (ClickHouse)

3. Verificación de factibilidad sobre el esquema real

Cada objetivo fue contrastado contra el esquema vigente de PostgreSQL para confirmar que los datos que lo sustentan existen. La siguiente tabla resume el soporte de datos identificado para cada informe:

INFORME	SOPORTE DE DATOS VERIFICADO
CONTROLAR VENTAS POR VENDEDOR (CUMPLIMIENTO INDIVIDUAL)	pedido.vendedor_id (FK directa a usuario, poblada desde el JWT en pedidos internos) + usuario
MONITOREAR PEDIDOS POR ESTADO (PENDIENTES, DESPACHADOS, ENTREGADOS)	pedido + estado_pedido
ANALIZAR TENDENCIA DE VENTAS POR MES Y CATEGORÍA	hechos de venta agregados por mes/categoría
MEDIR TICKET PROMEDIO DE COMPRA POR PERÍODO	agregación temporal de pedidos pagados
VER ÓRDENES DE COMPRA PENDIENTES DE APROBACIÓN	orden_compra (estado)
CONTROLAR CUENTAS POR PAGAR VIGENTES Y VENCIDAS	cuenta_por_pagar (estado, fecha_vencimiento, saldo_pendiente)
ANALIZAR GASTO DE COMPRA POR PROVEEDOR POR TRIMESTRE	facturas de compra agregadas por proveedor/trimestre
EVALUAR CUMPLIMIENTO DE TIEMPOS DE ENTREGA POR PROVEEDOR	orden_compra vs. recepcion_mercancia (fechas)
IDENTIFICAR PRODUCTOS BAJO STOCK MÍNIMO	inventario (stock_actual vs. stock_minimo)
CONSULTAR STOCK ACTUAL POR BODEGA	inventario + bodega
ANALIZAR ROTACIÓN DE INVENTARIO POR CATEGORÍA	kardex (movimiento_inventario) agregado por período
MEDIR PRODUCTOS DE BAJA/NULLA ROTACIÓN POR PERÍODO	kardex vs. catálogo, ventanas temporales
VER PEDIDOS PREPARADOS PENDIENTES DE DESPACHO	pedido (estado 'preparado')
CONSULTAR ENVÍOS POR TRANSPORTISTA	envio + transportista
ANALIZAR TIEMPO PROMEDIO DE ENTREGA POR TRANSPORTISTA/ZONA	envio (fecha_despacho vs. fecha_entrega_real) por zona
MEDIR TASA DE DEVOLUCIONES SOBRE DESPACHOS POR PERÍODO	devolucion vs. envio, series por período
VER TICKETS ABIERTOS POR PRIORIDAD Y ESTADO	ticket_soporte (prioridad, estado)
CONSULTAR TICKETS ASIGNADOS POR AGENTE	ticket_soporte (asignado_usuario_id) + usuario
ANALIZAR TIEMPO PROMEDIO DE RESOLUCIÓN POR CATEGORÍA POR MES	tickets cerrados agregados por categoría/mes
MEDIR CUMPLIMIENTO DE SLA POR PERÍODO	SLA derivado de prioridad vs. fecha de cierre
CONSULTAR ESTADO CONSOLIDADO DE PEDIDOS DEL DÍA	pedido + estado_pedido (fecha actual)
LISTAR CUPONES ACTIVOS Y PROMOCIONES VIGENTES (MARKETING)	cupon, promocion (activo, vigencia)
ANALIZAR TASA DE REDENCIÓN DE CUPONES E IMPACTO DE PROMOCIONES POR PERÍODO (MARKETING)	uso_cupon y descuentos agregados por período
ANALIZAR RENTABILIDAD POR CATEGORÍA POR PERÍODO	ventas vs. costo (producto_variante.costo) por categoría
MEDIR EVOLUCIÓN DE INGRESOS VS. COSTOS MENSUAL	factura_venta vs. factura_compra, serie mensual

4. Análisis de resultados

CLASIFICACIÓN	FUENTE	CANTIDAD	PORCENTAJE
INFORMES SIMPLES	BDR (PostgreSQL)	12	48%
INFORMES COMPUESTOS	BDColumnar (ClickHouse)	13	52%
TOTAL		25	100%

De los 25 informes tácticos identificados, 12 son simples y se resuelven directamente sobre la BDR PostgreSQL, mientras que 13 son compuestos y se procesan en la base columnar ClickHouse. La proporción es casi paritaria y no es casual: cada departamento necesita, por un lado, controlar el presente (colas de trabajo, pendientes, estados actuales — consultas puntuales que la BDR responde en milisegundos gracias a

sus índices y a su modelo normalizado) y, por otro, evaluar la evolución (tendencias, promedios por período, tasas y comparaciones entre meses o trimestres).

Los informes compuestos comparten tres rasgos que desaconsejan ejecutarlos sobre la base transaccional: (a) barren volúmenes históricos completos en lugar de filas puntuales; (b) exigen agregaciones (SUM, AVG, tasas) y agrupaciones por dimensiones de tiempo, categoría, proveedor o zona; y (c) su patrón de lectura masiva por columnas es exactamente el que un motor columnar optimiza. Ejecutarlos sobre PostgreSQL competiría por recursos con las transacciones operativas (ventas, checkout, kardex) y degradaría el sistema en producción.

Por ello, los 13 informes compuestos justifican el pipeline ETL hacia ClickHouse: un proceso periódico extrae los datos de PostgreSQL, los transforma (desnormalización, hechos y dimensiones, precálculo de métricas) y los carga en la base columnar, donde los análisis se ejecutan sin afectar la operación. La orquestación con Apache Airflow — programación, dependencias entre tareas, reintentos y monitoreo del pipeline — es precisamente la fase pendiente identificada en el proyecto.

5. Conclusión

El análisis del nivel táctico de RetailMind demuestra que las necesidades de dirección y control de los siete departamentos se cubren con 25 informes verificados contra el esquema real del sistema: 12 simples, atendidos por la base relacional PostgreSQL que ya soporta la operación, y 13 compuestos, que por su naturaleza agregada y temporal se destinan a la base columnar ClickHouse. La arquitectura híbrida existente resulta, por tanto, adecuada para el nivel táctico: la BDR responde el «ahora» de cada área y la BD columnar el «cómo evoluciona», quedando como siguiente paso natural la orquestación del pipeline ETL con Apache Airflow para automatizar la alimentación de los informes compuestos.